DS 1 Informatique - 18 septembre : 4h

Routage dans un réseau arborescent

Dans ce problème, on aborde une question soulevée dans la conception des réseaux : il s'agit d'affecter des adresses distinctes aux nœuds d'un réseau telle que la route entre deux nœuds puisse être déterminée par un calcul utilisant uniquement leurs deux adresses, sans référence à une connaissance globale du réseau. On s'intéresse plus particulièrement aux réseaux ayant une forme d'arbre.

Le problème est composé de trois parties différentes.

Dans tout le problème, un *n-arbre* est un arbre dont l'ensemble des *sommets* (on dit aussi parfois les nœuds) est $[\![0, n-1]\!]$, et dont la racine est 0. Chaque sommet a possède un *père* noté *pere[a]*. Par convention, la racine est son propre père ainsi *pere[0]=0*. L'ensemble des n-arbres est noté $\mathscr{A}(n)$.

L'ensemble des *ancêtres* du sommet a est constitué de a et des ancêtres de son père *pere[a]* ; la racine 0 est ainsi ancêtre de tous les sommets de l'arbre. Le *sous-arbre de racine a* noté A_a, est formé de tous les sommets de l'arbre A dont a est l'ancêtre.

Les *fils* d'un sommet a sont ainsi les $b \neq 0$ tels que *pere[b]=a*, (attention la racine n'est pas fils d'elle même). Une *feuille* est un sommet qui n'a pas de fils.

La *hauteur* (on dit aussi parfois profondeur) d'un sommet a notée $h(a)$ est un entier égale à 0 si a est la racine, elle est égale à la hauteur du père de a augmentée de 1 sinon. Le *plus proche ancêtre commun* à deux sommets a et b, noté *ppac(a,b)*, est le sommet de hauteur maximale qui est à la fois ancêtre de a et de b. Noter que si a est ancêtre de b alors *ppac(a,b)=a*.

Dans tout le problème, on considère que l'entier n est fixé, d'autre part pour un tableau `t`, `t[i]` dénoté l'indice d'élément i ; une liste contenant les entiers $x_0, x_1, \ldots, x_p$ est représentée par $\langle x_0, x_1, \ldots, x_p \rangle$; la liste vide est notée $\langle\rangle$.

On utilisera pour un n-arbre A :
- Un tableau de listes d'entiers `fils` de taille n tels que `fils[i]` est la liste des fils de i. On utilisera le type `int list array` pour cela.
- Un tableau d'entiers `pere` de taille n tel que `pere[i]` est le père de i. On utilisera le type `int array`.

1 Fonctions élémentaires

Question 1 : Calculer *ppac(i,j)* pour tous les couples de sommets de l'arbre A donné plus bas. On présentera le résultat sous forme d'un tableau de 8 lignes et 8 colonnes tel que la case sur la ligne i et la colonne j contienne le plus proche ancêtre de i et j.

Correction :

Dans le tableau ci-dessous, j'ai remplacé les 0 par des points pour ne pas le surcharger.

	0	1	2	3	4	5	6	7
0	.	.	.	.	.	.	.	.
1	.	1	4	.	4	4	.	.
2	.	4	2	.	4	2	.	.
3	.	.	.	3	.	.	.	.
4	.	4	4	.	4	4	.	.
5	.	4	2	.	4	5	.	.
6	.	.	.	.	.	.	6	6
7	.	.	.	.	.	.	6	7

Question 2 : Écrire une fonction `hauteur : int array -> int -> int` qui, étant donné le tableau des pères d'un arbre et un sommet a, calcule la hauteur de ce sommet.

Correction :
On utilise la définition récursive de la hauteur d'un sommet.

```
let rec hauteur peres i = match i with
    | 0 -> 0
    | i -> 1+hauteur (peres.(i))
```

Question 3 : Écrire une fonction `filsEnPere : int list array -> int array` qui, à partir d'un arbre donné par ses listes de fils, calcule le tableau des pères correspondant.

Correction :
On parcours les sommets puis leurs fils et l'on assigne à chaque fils son père. La fonction `remplir : int list -> int -> unit` met à jour le tableau des pères étant donné la liste `l` des fils du sommet `p`.

```
let filsEnPere fils =
    let peres = Array.make (Array.length fils) 0 in
    let rec remplir l p = match l with
        | [] -> ()
        | x::q -> peres.(x) <- p; remplir q p
    in
    for i = 0 to (Array.length fils) -1 do
        remplir (fils.(i)) i;
    done;
    peres;
```

Question 4 : Écrire une fonction `ppacMemeH : int array -> int -> int -> int` qui calcule le plus proche ancêtre commun de deux sommets a et b supposés de même hauteur dans un arbre pour lequel on donne le tableau des pères.

Correction :

```
let rec ppacMemeH peres a b =
    if a = b then a else ppacMemeH peres (peres.(a)) (peres.(b))
```

Question 5 : Justifier soigneusement la définition de la fonction `ppacMemeH` et analyser la complexité de celle-ci en fonction de $h(a)$.

Correction :
Montrons que la fonction `ppacMemeH` proposée est correcte. On cherche donc à montrer que pour deux sommets de même hauteur, la fonction renvoie bien leur plus proche ancêtre commun. On montre cela par induction.

- Si a et b son égaux, leur plus proche ancêtre commun est a. La fonction renvoie bien le bon résultat.
- Supposons maintenant que a b ne sont égaux. Par hypothèse d'induction, `peres.(a)` et `peres.(b)` étant de même hauteur, le résultat de `ppacMemeH peres (peres.(a))(peres.(b))` est bien leur plus proche ancêtre commun c. La fonction `ppacMemeH` renvoie alors c pour les sommets a et b. Puisque a et b sont distincts, leurs ancêtres communs sont aussi les ancêtres communs de leurs pères. c étant l'ancêtre commun de hauteur maximale de leurs pères, c'est aussi leur ancêtre commun de hauteur maximale. La fonction est donc correcte dans ce cas.

Par induction, on en déduit que la fonction est correcte.

Enfin, on peut remarquer que la hauteur des sommets décroît d'exactement 1 lors d'un appel récursif. Si on note $C(h)$ le temps d'exécution sur deux sommets de hauteur h, on a :

$$C(h) \leq \alpha + C(h-1)$$

On en déduit $C(h) \leq \alpha n = O(h)$.

Question 6 : En déduire une fonction `ppac : int array -> int -> int -> int` qui effectue le même calcul pour deux sommets a et b n'ayant pas nécessairement la même hauteur.

Correction :
On peut simplement chercher à rendre les deux sommets de la même hauteur en remplaçant le plus haut par son père. Cela ne change en effet pas les ancêtres communs. On fera enfin appel à `ppacMemeH` lorsque les deux sommets auront même hauteur.

```
let rec ppac peres a b =
    let ha = hauteur peres a in
    let hb = hauteur peres b in
    if ha = hb then
        ppacMemeH peres a b
    else
        if ha<hb then
            ppac peres a (peres.(b))
        else
            ppac peres (peres.(a)) b
```

Cette solution présente un défaut important : elle répète des appels aux fonctions `hauteur` lors de chaque appel récursif. On peut l'améliorer à l'aide d'une fonction auxiliaire `remonter : int -> int -> int -> int ->int` qui, étant donnés deux sommets et leurs hauteurs, remonte les pères du plus haut des deux jusqu'à ce que les hauteurs soient égales et exécute alors `ppacMemeH` sur ceux-ci.

```
let rec remonter a ha b hb =
    if ha = hb then
        ppacMemeH peres a b
    else
        if ha<hb then
            remonter peres a  ha (peres.(b)) (hb-1)
        else
            remonter peres (peres.(a)) (ha-1) b hb
```

La fonction `ppac` consiste alors en un appel à `remonter` avec les hauteurs correctement initialisées.

```
let ppac peres a b = remonter a (hauteur peres a) b (hauteur peres b)
```

2 Arbres binaires complets

Un *arbre binaire complet* est un arbre dans lequel tout sommet qui n'est pas une feuille a exactement deux fils, et tel que toutes les feuilles sont à la même hauteur. On note $\mathscr{B}(h)$ l'ensemble des n-arbres binaires complets dans lesquels les feuilles sont à hauteur h. Un exemple d'arbre binaire complet B est donné à droite de la figure 1.

Question 7 : Déterminer pour $0 \leq k \leq h$ le nombre s_k de sommets de hauteur k dans un arbre B de $\mathscr{B}(h)$, justifiez votre réponse. En déduire le nombre n de sommets d'un arbre de $\mathscr{B}(h)$ en fonction de h.

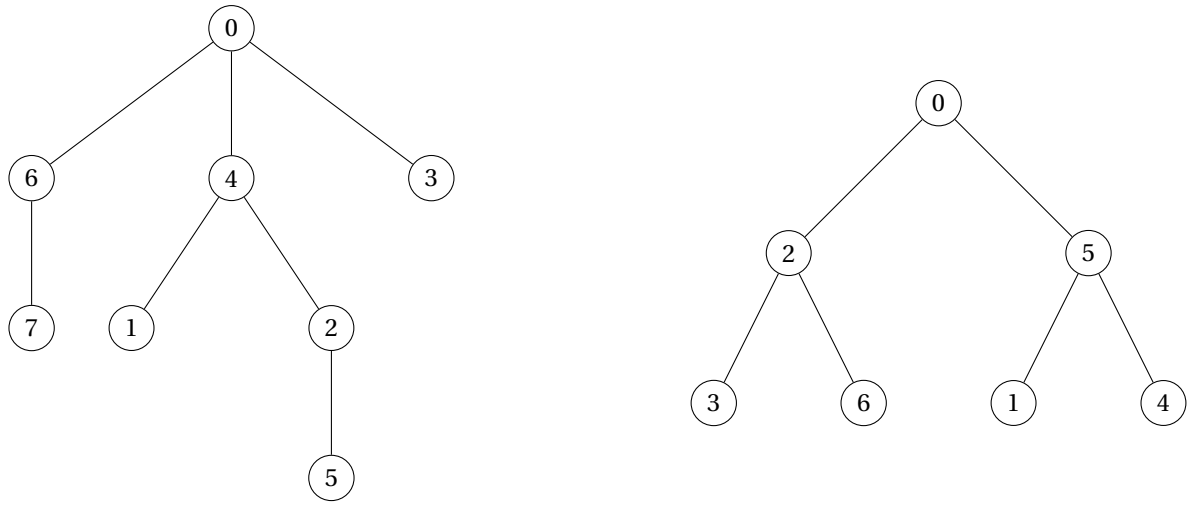

FIGURE 1 – Un arbre $A \in \mathscr{A}(8)$ et un arbre binaire complet $B \in \mathscr{B}(2)$.

Correction :
On sait que $s_0 = 1$ car seule la racine est de hauteur 0. Puisque chaque sommets a exactement deux fils, on a aussi $s_{k+1} = 2 \times s_k$. On en déduit par récurrence : $s_k = 2^k$. Enfin, le nombre de sommets d'un arbre de $\mathscr{B}(h)$ est la somme du nombre de sommets de chaque hauteur :

$$n = \sum_{k=0}^{h} 2^k = 2^{h+1} - 1$$

On aura reconnu la somme des termes d'une suite géométrique.

Dans la suite, B est un arbre de $\mathscr{B}(h)$ ayant n sommets ; à chaque sommet a de B, on associe une étiquette $l(a) \in [\![1, n]\!]$ satisfaisant la condition suivante :
Pour chaque sommet a ayant pour liste de fils $\langle b, c \rangle$:

$$\max_{u \in B_b} l(u) < l(a) < \min_{v \in B_c} l(v)$$

Question 8 : Calculer les valeurs de $l(a)$ pour les 7 sommets de l'arbre binaire complet représenté sur la droite de la figure 1 : l'ordre des fils dans les listes correspond à la lecture de gauche à droite de la figure.

Correction :
Le tableau suivant indique des valeurs de $l(a)$ qui conviennent :

0	1	2	3	4	5	6
4	5	2	1	7	6	3

Question 9 : Démontrer l'unicité et l'existence de la fonction d'étiquetage pour tout arbre.

Correction :
Prouvons l'unicité de l. Soit l_1 et l_2 deux fonctions des sommets de B dans $[\![1,]\!]n$ qui satisfont la propriété. Pour cela, on commence par montrer que l_1 est injective : soient deux sommets a, b distincts et $c = ppac(a, b)$. Puisque $a \neq b$, on a $a \neq c$ ou $b \neq c$. Considérons, sans perte de généralité que $a \neq c$. c est donc un sommet interne puisqu'il est ancêtre strict de a. a appartient l'une des deux branches issues de c. Si a appartient à la branche gauche issue de c alors b n'appartient pas à cette branche par définition de c. On distingue alors deux cas :

- $b = c$ Dans ce cas, d'après la propriété, $l_1(a) < l_1(c) = l_1(b)$.
- $b \neq c$ Dans ce cas, $l_1(a) < l_1(c) < l_1(b)$ puisque b appartient à la branche droite.

On conclut de même si a appartient à la branche droite issue de c. L'injectivité de l_1 est ainsi prouvée, et elle implique sa bijectivité puisque $|B| = |[\![1, n]\!]$. Par symétrie, l_2 est aussi bijective. On considère enfin $l_3 : x \mapsto$

$\max(l_1(x), l_2(x))$. l_3 vérifie alors la propriété :

$$\max_{u \in B_b} l_3(u) < l_3(a) < \min_{v \in B_c} l_3(v)$$

pour tout sommet a de fils gauche b et de fils droit c, donc l_3 est aussi bijective. Puisque $l_3 \geq l_1$, ceci implique $l_3 = l_1$ d'où finalement $l_1 = l_2$.

Pour prouver l'injectivité, il suffit de remarquer qu'un parcours infixe de l'arbre binaire construit un étiquetage qui convient.

Question 10 : En déduire une fonction `etiquettes : int list array -> int array` qui calcule les étiquettes $l(a)$ pour tous les sommets d'un arbre $\mathscr{B}(h)$ donné par ses listes de fils.

Correction :
On réalise un parcours infixe. La fonction `parcours : int -> int -> int` prend deux entiers s et k et réalise un parcours de l'arbre enraciné en s. k est le numéro à affecter au premier sommet visité dans `etiq`. La fonction renvoie le numéro à affecter au prochain sommet visité.

```
let etiquettes fils =
    let etiq = Array.make (Array.length fils) 0 in
    let rec parcours s k = match fils.(k) with
        | [] -> etiq.(s) <- k; k+1
        | g::d::[] -> let k1 = parcours g k in
                      etiq.(s) <- k1;
                      parcours d (k1+1)
    in
    let n = parcours 0 1 in etiq
```

Question 11 : Un arbre B de $\mathscr{B}(h)$ de racine 0 ayant pour liste de fils $\langle a, b\rangle$ se décompose en deux sous-arbres B_a et B_b. Déterminer les valeurs de $l(0)$, de $l(a)$ et de $l(b)$ pour les fils a et b de la racine.

Correction :
$l(0)$ est le numéro de la racine dans un parcours infixe. C'est donc la taille de son fils gauche plus 1 soit 2^h. $l(a)$ le numéro de la racine pour un arbre d'une hauteur d'un de moins. On a donc $l(a) = 2^{h-1}$. Enfin, $l(b) = 2^h + 2^{h-1}$. En effet, c'est le numéro de la racine d'un arbre binaire complet de hauteur $h-1$ en commençant à numéroter à partir de $l(0) = 2^h$.

Question 12 : Démontrer que si a et b sont deux sommets quelconques de B, alors *r=l(ppac(a,b))* est déterminé de manière unique à partir de $p = l(a)$ et $q = l(b)$ par la propriété suivante : r est parmi les entiers compris (au sens large) entre p et q celui possédant le plus grand diviseur qui soit une puissance de 2.

Correction :
Soit un sommet s de hauteur h_s. Montrons d'abord que $l(s)$ est un multiple de 2^{h-h_s} et n'est pas divisible par 2^{h-h_s+1}. On le montre par récurrence sur h_s.

- La racine a pour étiquettes 2^h. C'est bien divisible par 2^h mais pas par 2^{h+1}.
- Supposons que la propriété soit vraie à la hauteur h_s. SOit s un sommet de hauteur $h_s + 1$. Son père r est de hauteur h_s et vérifie la propriété par hypothèse de récurrence. On suppose que s est le fils gauche de r. Par le même raisonnement qu'à la question précédente, on a $l(r) = l(s) + 2^{h-h_s-1}$. Par l'hypothèse de récurrence, on en déduit que s la vérifie aussi. La preuve est identique dans le cas où s est le fils droit de r.

Supposons, sans perte de généralité $p \leq q$. Or, parmi les éléments étiquetés entre p et q, r est aussi le plus haut. Il s'agit donc bien de celui possédant le plus grand diviseur qui soit une puissance de 2.

Question 13 : Écrire une fonction récursive `mu : int -> int -> int` qui, détermine *r=l(ppac(a,b))* à partir de $p = l(a)$ et $q = l(b)$; on supposera que $p \leq q$. La fonction doit être de complexité $O(\log_2(q-p))$.

Correction :
D'après la question précédente, r est l'élément de $[\![p,q]\!]$ qui est divisible par la plus grande puissance de deux. On note 2^k la plus grande puissance de 2 divisant r. On a alors : $\lceil \frac{p}{2^k} \rceil \leq \lfloor \frac{q}{2^k} \rfloor$. On en déduit $\lceil \frac{p}{2^k} \rceil \leq \lfloor \frac{q}{2^k} \rfloor$ vu le caractère maximal de k, et $r = 2^k \lceil p/2^k \rceil$. On en déduit une fonction récursive :

```
let rec mu p q = if p=q then p else 2*(mu ((p+1)/2) (q/2))
```

À chaque étape, la différence $p-q$ est divisée par 2, ce qui fait que la profondeur de la récursion est proportionnelle à $\log_2(p-q)$. La fonction a donc bien une complexité $\log_2(p-q)$.

3 Arbres généraux

Dans cette partie, on utilise les définitions et notations suivantes :

Dans un arbre A, le poids $w[a]$ d'un sommet a est le nombre de sommets du sous arbre A_a de racine a, ainsi $w[a]=1$ si et seulement si a est une feuille, noter que $w[0]=n$ (c'est le nombre de sommets de A).

Un arbre A est dit *gauche* si pour chaque sommet a (qui n'est pas une feuille) le premier élément de la liste est de poids supérieur ou égal à celui de tous les autres sommets de cette liste. Dans un arbre gauche, le premier élément de la liste des fils d'un sommet est dit *lourd*, les autres sommets sont dits *légers*. La racine est un sommet léger.

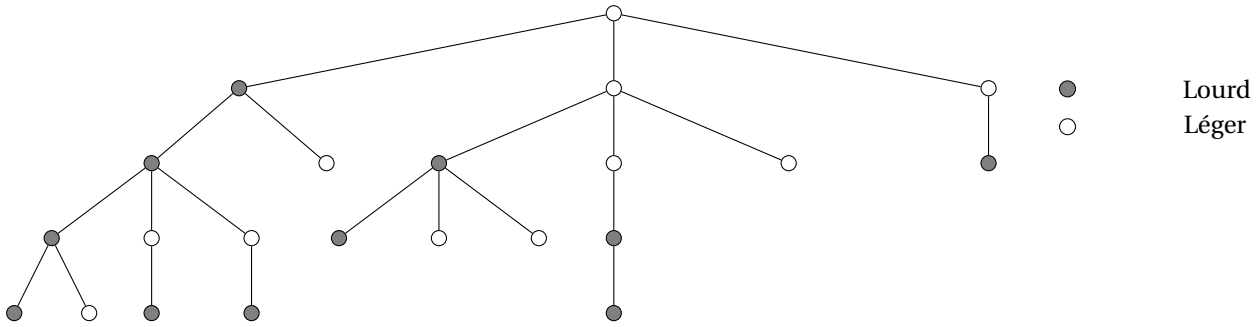

FIGURE 2 – Sommets lourds et légers

Question 14 : Écrire une fonction `poids : int list array -> int array` qui calcule les poids de tous les sommets donné par ses listes de fils.

Correction :
On parcourt l'arbre en profondeur d'abord, en mettant à jour le poids d'un sommet entre chaque visite d'un fils. La fonction `calcule : int -> int list -> unit`, étant donné un sommet s et une liste l, calcule le poids de tous les sommets de l'arbre enraciné en s. La liste l correspond aux fils de s non examinés.

```
let poids fils =
    let n = Array.length fils in
    let poids = Array.make n 1 in
    let rec calcule a l = match l with
        | [] -> ()
        | b::q -> calcule b fils.(b); p.(a) <- p.(a) + p.(b); calcule a
            q
    in calcule 0 fils.(0);
    p
```

Question 15 : Écrire une fonction `gauchir : int list array -> int array -> int list array` qui calcule un arbre gauche obtenu en réordonnant les fils de chaque sommet de façon telle que le premier fils soit de poids

supérieur ou égal à celui des autres. Sont donnés dans cette fonction les listes des fils et le tableau des poids des sommets de l'arbre.

Correction :

On réalise ici aussi un parcours de l'arbre au cours duquel on isole pour chaque sommet interne un fils de poids maximal pour le placer en tête de la liste des fils. La fonction `reordonne : int list -> int list` reçoit en paramètre une liste de fils d'un sommet et renvoie la même liste mais commençant par un sommet lourd.

```
let gauchir fils poids =
    let rec reordonne l = match l with
        | [] -> []
        | a::q -> match reordonne q with
            | [] -> [a]
            | b::reste if w.(b) > w.(a) then b::a::reste else l
    in
    let n = Array.length fils in
    let filsG = Array.make n [] in
    for i=0 to n-1 do
        filsG.(i) <- reordonne fils.(i)
        done;
    filsG
```

Question 16 : Soit a un sommet léger d'un arbre gauche A et b son père, montrer que $w[b] > 1w[a]$. En déduire que le nombre d'ancêtres légers de a est inférieur à $1+\log_2 n$.

Correction :

Soit c le fils gauche de b. Puisque l'arbre est gauche, on a $w[c] \geq w[a]$. Or, par définition du poids, $w[b] \geq w[c]+w[a]+1$. On en déduit bien $w[b] < 2w[a]$. Soient $a = a_0, a_1, \ldots, a_k$ les ancêtres légers de a, classés par hauteur décroissante. Soit pour $i < k$, b_i le père de a_i. On a $w(a_{i+1}) \geq w(b_i) > 2w(a_i)$, d'où $w(a_k) > 2^k w(a)$. Or $w(a_k) \leq n$ et $w(a) \geq 1$, donc $2^k < n$ c'est-à-dire $k < \log_2(n)$. Ainsi le nombre d'ancêtres légers de a est $1+k < 1+\log_2(n)$.

On utilise dans toute la suite la notion de *cime* d'un sommet.

- Si a est léger, *cime[a]* est égale à *pere[a]*.
- Si a est lourd, *cime[a]* est le plus proche ancêtre de a qui est léger.

Ainsi, en raison de nos conventions, *cime[0]=0*. On appelle A' l'arbre gauche obtenu en appliquant la fonction `gauchir` à l'arbre A de la figure 1. Les cimes des sommets de l'arbre A' sont alors donnés par le tableau suivant :

i	0	1	2	3	4	5	6	7
cime	0	4	0	0	0	0	0	6

Question 17 : Écrire une fonction `cimes : int list array -> int array` qui calcule les sommets cimes de tous les sommets à partir des listes de fils d'un arbre gauche.

Correction :
On utilise une fonction `calcule : int -> bool -> int list -> bool -> unit()`. Celle-ci reçoit en paramètre un sommet a, un booléen indiquant si a est léger, la liste des fils de a non traités ainsi qu'un booléen indiquant si le sommet lourd des fils de a a déjà été traité.

```
let cimes fils =
    let cime = Array.make (Array.length fils) 0  in
    let rec calcule a a_leger l leger = match l with
        | [] -> ()
        | b::q -> cime.(b) <- if a_leger or leger then a else cime.(a);
                        (* Si b est léger, a est son père *)
                        (* Sinon, si a est léger, c'est le plus proche
                           ancêtre léger de b *)
                        calcule b leger fils.(b) false;
                        calcule a a_leger q true
    in calcule 0 true fils.(0) false;
    cime
```

On se propose d'attribuer des étiquettes aux sommets d'un arbre gauche A. Pour cela, on commence par construire deux tableaux d'entiers *t1* et *t2* associés aux sommets.

Le tableau $t1$ est tel que *t1[0]=0* et *t1[a]=i* si a est le i-ème élément de la liste des fils de son père.

Le tableau $t2$ est tel que *t2[a]=0* si a est léger et *t1[a]=t2[pere[a]]+1* si a est lourd.

Les étiquettes $\lambda(a)$ des sommets sont alors des listes d'entiers construites de la façon suivante :

$$\lambda(0) = \langle\rangle \qquad \text{et pour} \quad a \neq 0 \;\; \lambda(a) = \lambda(cime[a]) \bullet \langle \mathrm{t1[a]}, \mathrm{t2[a]} \rangle$$

dans cette formule, $\bullet$ dénote la concaténation des listes.

Question 18 : Donner l'arbre A' défini entre les questions 16 et 17 ; puis donner les valeurs *t2[a]* pour tous les sommets a ; calculer enfin leurs étiquettes $\lambda(a)$.

Correction :

On obtient le tableau suivant :

a	0	1	2	3	4	5	6	7
$t_2[a]$	0	0	2	0	1	3	0	1
$\lambda(a)$	$\langle\rangle$	$\langle 0,1,1,0\rangle$	$\langle 0,2\rangle$	$\langle 2,0\rangle$	$\langle 0,1\rangle$	$\langle 0,3\rangle$	$\langle 1,0\rangle$	$\langle 1,0,0,1\rangle$

Question 19 : Écrire une fonction `etiquettes : int list array -> int array -> int list array` qui calcule les étiquettes des sommets d'un arbre gauche donné par ses listes de fils et pour lequel on donne aussi le tableau des cimes des sommets. On pourra utiliser l'opérateur `@` qui calcule la concaténation de deux listes.

Correction :
On peut remarquer pour cette fonction que `t1` indique simplement la position du sommet dans dans la liste tandis que `t2` indique la distance au plus proche ancêtre léger. Une fois ces remarques réalisées, la fonction devient assez proche de celle utilisée pour calculer les cimes de chaque sommet puisque les tableaux `t1` et `t2` peuvent être construit au cours du parcours. Cette fois-ci la fonction `calcule_bis : int -> int list -> int -> int -> unit` reçoit en paramètre un sommet a, la liste de ses fils qu'il reste à traiter, le nombre de sommets déjà défilés ainsi que la distance de a à son plus proche ancêtre léger.

```
let etiquettes fils cimes =
    let etiq = Array.make (Array.length fils) [] in
    let rec calcul_bis a l i distance_leger_a = match f with
        | [] -> ()
        | b::q -> let distance_leger_b = (if i = 0 then
            distance_leger_a+1 else 0) in
                    etiq.(b)<-lambda.(cime.(b))@[i;distance_leger_b];
                    calcul_bis b (fils.(b)) 0 distance_leger_b;
                    calcul_bis a q (i+1) distance_leger_a
    in calcul 0 fils.(0) 0 0;
    etiq
```

On aurait aussi pu simplement calculer les tableaux `t1` et `t2` puis en déduire λ.

Question 20 : Écrire une fonction `trouve : int list array -> int list -> int` qui, à partir de l'étiquette d'un sommet d'un arbre gauche A et des listes de fils des sommets de cet arbre, détermine ce sommet. On justifiera soigneusement sa correction.

Correction :
Pour écrire la fonction `trouve`, on remarque la propriété suivante : Soit a un sommet et $b = cime(a)$. On a deux cas :

- Si `t2[a]=0` alors a est le `t1[a]`-ième fils de b.
- Sinon, a est le `t2[a]`-ième descendant gauche de b.

Cette propriété se prouve par récurrence sur la hauteur de a.

Pour calculer a tel que $\lambda(a) = \langle u_1, v_1, \ldots, u_k, v_k \rangle$, on cherche sa cime b d'étiquette $\langle u_1, v_1, \ldots, u_{k-1}, v_{k-1} \rangle$. Une fois celle-ci trouvée, on a deux cas :

- Si $v_k = 0$, a est le u_k-ième fils de b.
- Sinon, a est le u_k-ième descendant gauche de b.

On introduit donc les fonctions récursives suivantes :

- `ieme_fils : int list -> int -> int` qui, étant donnés une liste l et un indice i, renvoie le i-ème élément de l.
- `ieme_descendant : int list array -> int -> int -> int` qui, étant donnés un arbre, un sommet s de celui-ci et un entier i, renvoie le i-ème descendant gauche de s.
- `calcul : int -> int list` qui, étant donné un sommet b et la partie d'étiquette restant à traiter, calcule le sommet auquel elle correspond.

```
let rec ieme_fils l i = match i,l with
    | 0, h::q -> h
    | i, h::q -> i_eme_fils q (i-1)
    | _,[] -> failwith "liste trop courte"

let rec ieme_descendant fils s i = match i,fils.(s) with
    | 0,_ -> s
    | i,h::q -> i_eme_descendant fils h (i-1)
    | _,_ -> failwith "arbre trop court"

let rec trouve fils etiq =
    let rec calcul b etiq = match etiq with
        | [] -> b
        | i::0::q -> calcul (ieme_fils (fils.(b)) i) q
        | _::i::q -> calcul (ieme_descendant fils b i) q
    in calcul 0 etiq
```

Question 21 : Montrer que, pour tout sommet a d'un arbre, la longueur de l'étiquette $\lambda(a)$ est majorée par 4 fois le nombre de ses ancêtres légers.

Correction :
Soit A un arbre et a un sommet de cet arbre. On définit la suite a_n de la façon suivante :

- $a_0 = a$
- $a_{k+1} = cime(a_k)$

On note K le premier entier tel que $a_k = 0$.

Soit α (respectivement β) le nombre de sommets légers (respectivement lourds) dans $(a_0, \dots a_K)$. On note γ le nombre d'ancêtres légers de a.
On a $\alpha \geq \gamma$. De plus, puisque la cime d'un sommet lourd est toujours un sommet léger, chaque sommet lourd de la suite est suivi d'un sommet léger. On a donc aussi $\beta \leq \alpha$.
La taille de l'étiquette de a est d'exactement $2(K-1)$ puisque pour chaque cime élément différent de 0, on ajoute deux valeurs à la l'étiquette. En remarquant que $K = \alpha + \beta$, on obtient donc :

$$|\lambda(a)| = 2(K-1) = 2(\alpha+\beta-1) \leq 2(2\gamma-1) \leq 4\gamma$$

Cela conclut la preuve recherchée.

Question 22 : Expliquer comment calculer $\lambda(ppac(a,b))$ à partir des deux listes $u = \lambda(a)$ et $v = \lambda(b)$.

Correction :
Soit w le plus long préfixe de longueur paire commun entre les étiquettes u et v. On note $u = w \bullet u'$ et $v = w \bullet v'$.
Alors :

- Si $u' = \langle 0, i\rangle \circ u''$ et $v' = \langle 0, j\rangle \circ v''$ avec alors $\lambda(ppac(a,b)) = w \circ \langle 0, \min(i,j)\rangle$. Cela correspond au cas où le dernier sommet de l'étiquette en commun est un sommet léger. Il faut alors savoir quel est le dernier descendant gauche à partir duquel diffèrent les chemins menant à u et v d'où $\min(i,j)$.
- Sinon, $\lambda(ppac(a,b)) = w$. Cela correspond au cas où la différence se fait parmi les fils de w.

Question 23 : Écrire une fonction `ppac2 : int list array -> int list array -> int -> int ->` qui, étant donné les listes de fils des sommets d'un arbre gauche, le tableau des étiquettes des sommets ainsi que deux sommets a et b, calcule $ppac$(a,b).

Correction :
On introduit une fonction `trouve_w : int list -> int list -> int list` qui, étant donnés u et v, renvoie le w associé.

```
let ppac2 fils etiquettes a b =
    let rec trouve_w u v = match u, v with
        | [], [] -> []
        | x1::y1::q1, x2::y2::q2 when x1 = x2 && y1 = y2 -> x1 :: y1 ::
            (trouve_prefixe_commun q1 q2)
        | 0::i::q1 , 0::j::q2 -> [0; if i < j then i else j]
        | _ -> []
    in trouve (trouve_w (etiquettes.(a)) (etiquettes.(b)))
```

Question 24 : Analyser la complexité de la fonction `ppac2`.

Correction :
La complexité de `trouve_w` est proportionnelle à la taille des étiquettes u et v. D'après les questions 16 et 21, celle-ci est en $O(\log n)$. La complexité de `trouve` est proportionnelle à la hauteur de $ppac(a,b)$. La complexité de `ppac2` est majorée par la somme des deux.